

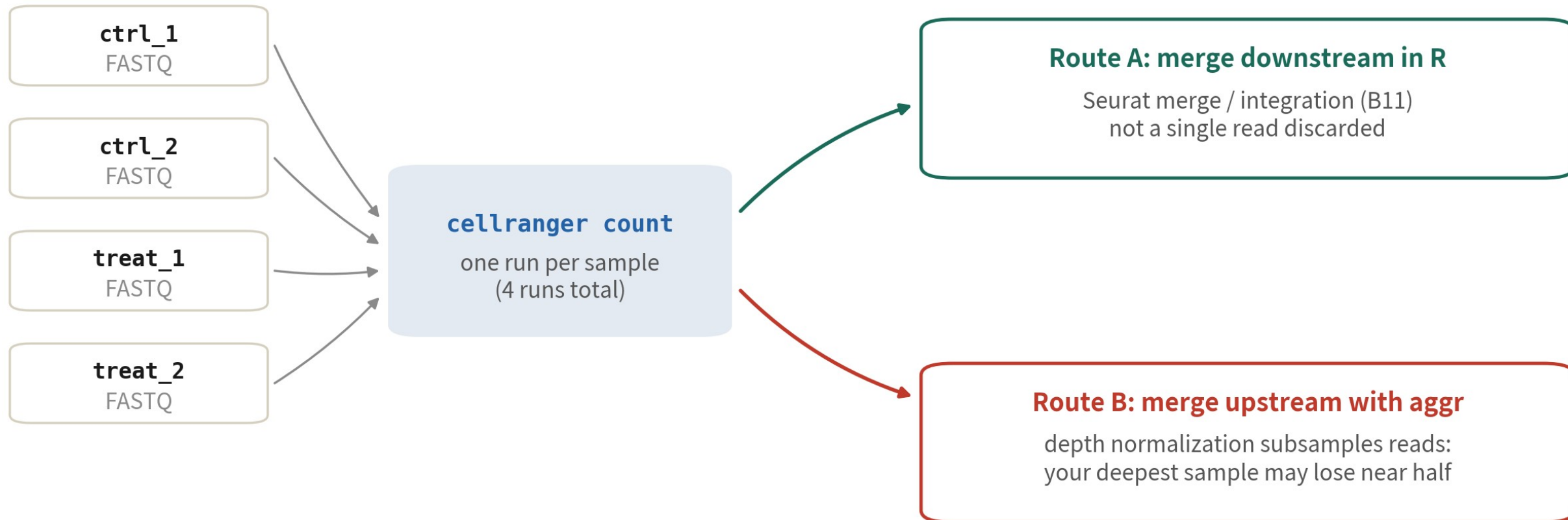
B4 Series B

Multi-sample & multimodal: aggr, multi, Feature Barcode

Read multi-sample and CITE-seq / hashing outputs with confidence — and pick the right command.

About 35 min | Prerequisites: B3 | Data: 10x public multi / hashing examples (read-only)

Four samples, two roads



Four samples to combine — which route, and at what cost?

Both roads combine four samples into one analysis — they differ in what happens to your reads

WHERE THIS EPISODE STARTS

**Pick the wrong way to combine,
and the bill arrives weeks later.**

aggr silently subsamples reads away — and they do not come back. Most people find out the day they need full depth.

What you will learn

- 1 Tell count, aggr and multi apart — and defend when to use which
- 2 Describe what Feature Barcode data (ADT / HTO) looks like in the matrix
- 3 Read the three hashing demux calls: singlet, doublet, negative

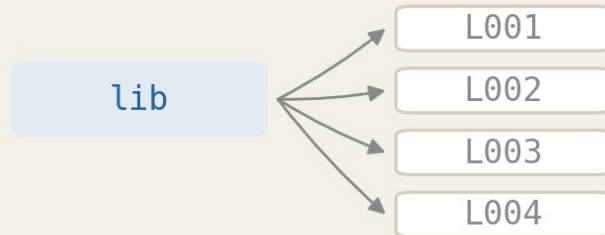
01

Three kinds of 'multi' — sort them first

Lanes, samples, modalities — each handled differently

Lanes, samples, modalities

multiple lanes



one library split across lanes;
count merges them automatically (B2)

nothing to do

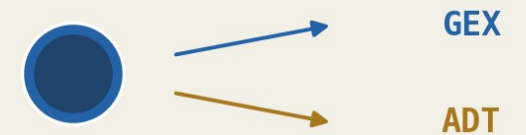
multiple samples



different donors / conditions;
run count per sample, then combine

this episode: aggr vs R

multiple modalities



same cells, two libraries:
RNA + surface protein (Feature Barcode)

this episode: multi / FB

Lanes are a non-problem — count merges them itself; the other two are today's topic

Combining samples: upstream or downstream?

This is a strategy call, not a syntax call

downstream

count per sample, merge in R

- keeps every read and all information
- batch effects handled by integration (B11)
- the modern default route

upstream

cellranger aggr into one bundle

- one matrix, one web_summary
- default depth normalization discards reads
- fine for quick overviews or single-matrix tools

CORE IDEA

**Merging matrices is easy —
merging comparability is the hard part.**

aggr buys comparability by discarding reads; integration buys it with statistics. B11 explains why the latter won.

02

Running cellranger aggr

One CSV, one command, one parameter to think hard about

The aggregation CSV: two columns

```
cat> aggr.csv << 'EOF'  
sam ple_id,m olecule_h5  
ctrl_1,rns/ctrl_1/outs/m olecule_info.h5  
ctrl_2,rns/ctrl_2/outs/m olecule_info.h5  
treat_1,rns/treat_1/outs/m olecule_info.h5  
treat_2,rns/treat_2/outs/m olecule_info.h5  
EOF
```

aggr reads each run's molecule_info.h5 — not the matrix folder

Run aggr: --normalize is the decision

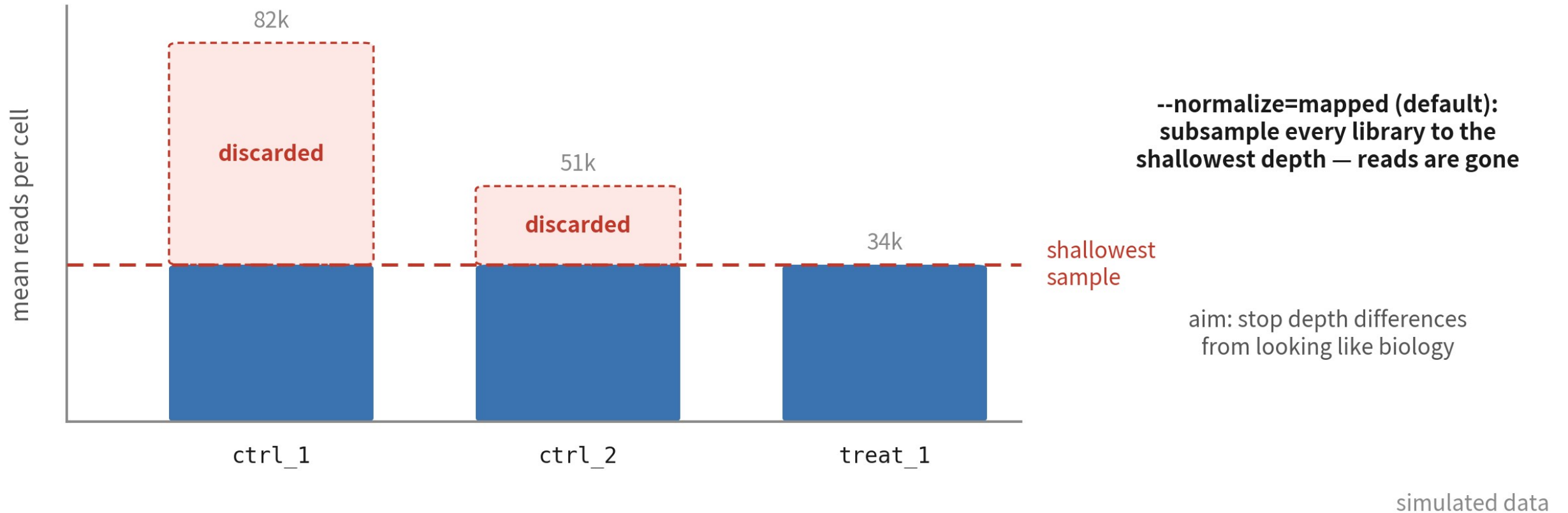
```
cellranger aggr \  
  --id= all_samples \  
  --csv= aggr.csv \  
  --normalize=mapped # 預設。 none = 保留全部 reads
```

Outputs:

- Run summary HTML: `/.../all_samples/outs/web_summary.html`
- Aggregation CSV: `/.../all_samples/outs/aggregation.csv`
- Filtered feature-barcode matrices:
`/.../outs/count/filtered_feature_bc_matrix`

mapped = subsample all runs to equal depth; none = keep everything

What depth normalization does



Race to the bottom: every run is subsampled to the shallowest one; the excess reads are discarded

aggr output: barcodes gain a suffix

aggregation CSV

sample_id,molecule_h5

ctrl_1,...molecule_info.h5

ctrl_2,...molecule_info.h5

treat_1,...molecule_info.h5

treat_2,...molecule_info.h5

input is molecule_info.h5,
not the matrix folder

cellranger
aggr

one combined matrix

AAACCCAAGAAACACT -1

TGCACGGTCAGGACGT -1

AAACCCAAGAAACACT -2

CTTCAATTCGGAGATG -3

suffix -1 / -2 / -3 / -4:
row number in the CSV

the same barcode sequence can
appear in several samples —
only the suffix separates them

-1, -2, -3 ... map to CSV row order; downstream tools use the suffix to trace each cell's sample

aggr: the verdict

When it still helps



you want one
overview
web_summary, or a
tool that only takes
one matrix

When you do use it



decide --normalize
deliberately; none is
often right for
downstream work

The mainstream route



count per sample,
merge in R, integrate
in B11

03

What cellranger multi is

One run, several libraries, several samples — one conductor

count or multi?

GEX only

one library per sample

cellranger count

the B3 world

GEX + antibodies (ADT / HTO)

add a feature reference

count --libraries / multi

both roads work

CellPlex multiplexing (CMO)

needs per-sample demux

cellranger multi

multi only

5' VDJ (TCR / BCR)

GEX and VDJ must pair

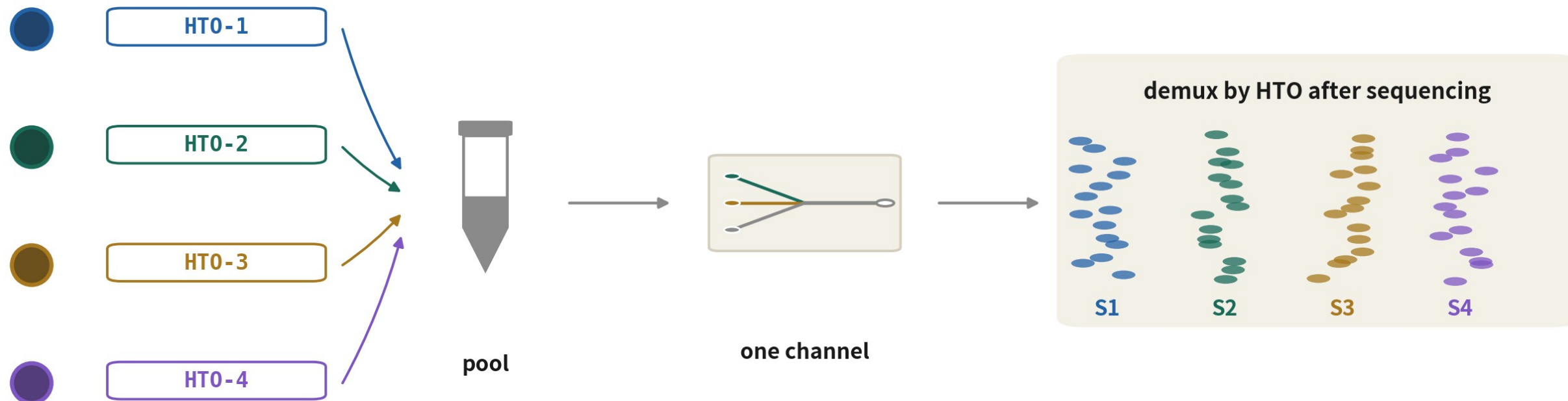
cellranger multi

multi only

rule: per-sample demux or VDJ involved → multi; otherwise count suffices

Anything needing per-sample demux, or involving VDJ, must go through multi

Why multiplex at all



label each sample with a distinct hashtag antibody

wins: cheaper, zero batch effect within the pool, cross-sample doublets detectable

Tag first, pool, run one channel — then split samples back out by their tags

The multi config (1): resources & libraries

```
# multi_config.csv — 一個檔案分好幾個 [區塊]  
[gene-expression]  
reference,refs/refdata-gex-GRCh38-2024-A  
create-bam,false  
  
[libraries]  
fastq_id,fastqs,feature_types  
pbmc_gex,fastq/gex,Gene Expression  
pbmc_cmo,fastq/cmo,Multiplexing Capture
```

The libraries block declares what each FASTQ set is — here GEX plus CellPlex CMO

The multi config (2): the samples block

```
[samples]
sample_id,cm o_ids
donor_A,CM 0 301
donor_B,CM 0 302
donor_C,CM 0 303
donor_D,CM 0 304
```

```
$ cellranger multi --id=pbm_c_multi --csv=multi_config.csv
...
Outputs:
- per-sample outputs: /.../pbm_c_multi/outs/per_sample_outs
```

The samples block maps tags to samples — omit it and nothing can be demultiplexed

multi output: per_sample_outs

pbmc_multi/outs/

└─ multi/count/raw_feature_bc_matrix/

← all cells, not yet split

└─ **per_sample_outs/**

│ └─ **donor_A/**

│ │ └─ web_summary.html

← one report per sample

│ │ └─ **count/sample_filtered_feature_bc_matrix/**

← this is what downstream reads

│ └─ **donor_B/ ...**

│ └─ **donor_C/ ...**

└─ config.csv

← your config is archived here

count keeps one folder per run; multi splits one run into per_sample_outs

One folder and one web_summary per sample; downstream reads sample_filtered_feature_bc_matrix

multi: what this section covered

One config



three blocks to know:
gene-expression,
libraries, samples

Demux built in



CellPlex sample
assignment happens
upstream

Per-sample outputs



per_sample_outs
feeds straight into B5's
loading step

04

Feature Barcode: turning protein into sequence

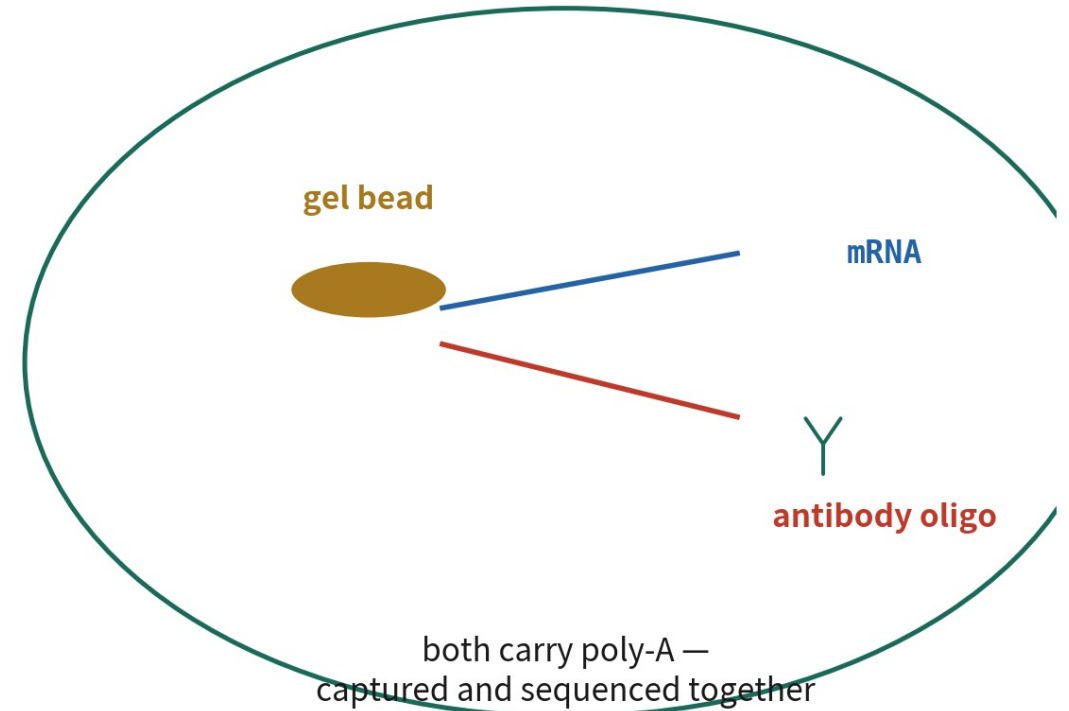
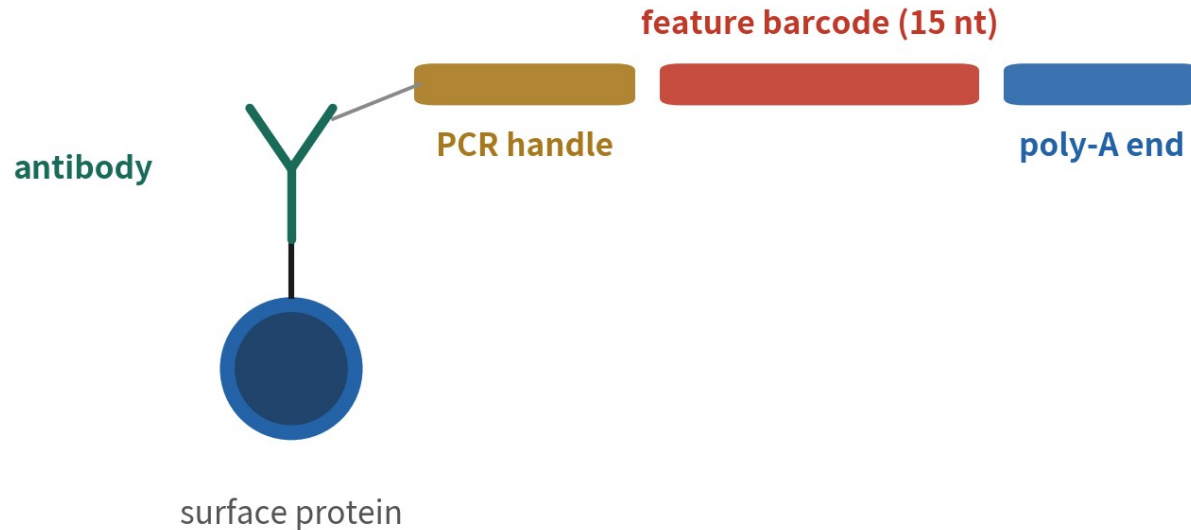
CITE-seq ADT and cell hashing HTO share one trick

One oligo makes an antibody sequenceable

ADT = protein signal (CITE-seq, Stoeckius 2017)

HTO = sample tag (cell hashing, Stoeckius 2018)

an oligo tied to the antibody —
the middle stretch is its ID



The antibody carries a synthetic oligo — reading its barcode is 'seeing' the protein

count can run FB too: the libraries CSV

```
cat> libraries.csv << 'EOF'
fastqs,sample,library_type
fastq/gex,pbm c_gex,Gene Expression
fastq/adt,pbm c_adt,Antibody Capture
EOF
```

```
cellranger count --id=pbm c_citeseq \
  --transcriptome=refs/refdata-gex-GRCh38-2024-A \
  --libraries=libraries.csv \
  --feature-ref=feature_ref.csv \
  --create-bam=false
```

No --fastqs here: both libraries go into libraries.csv, each with its type

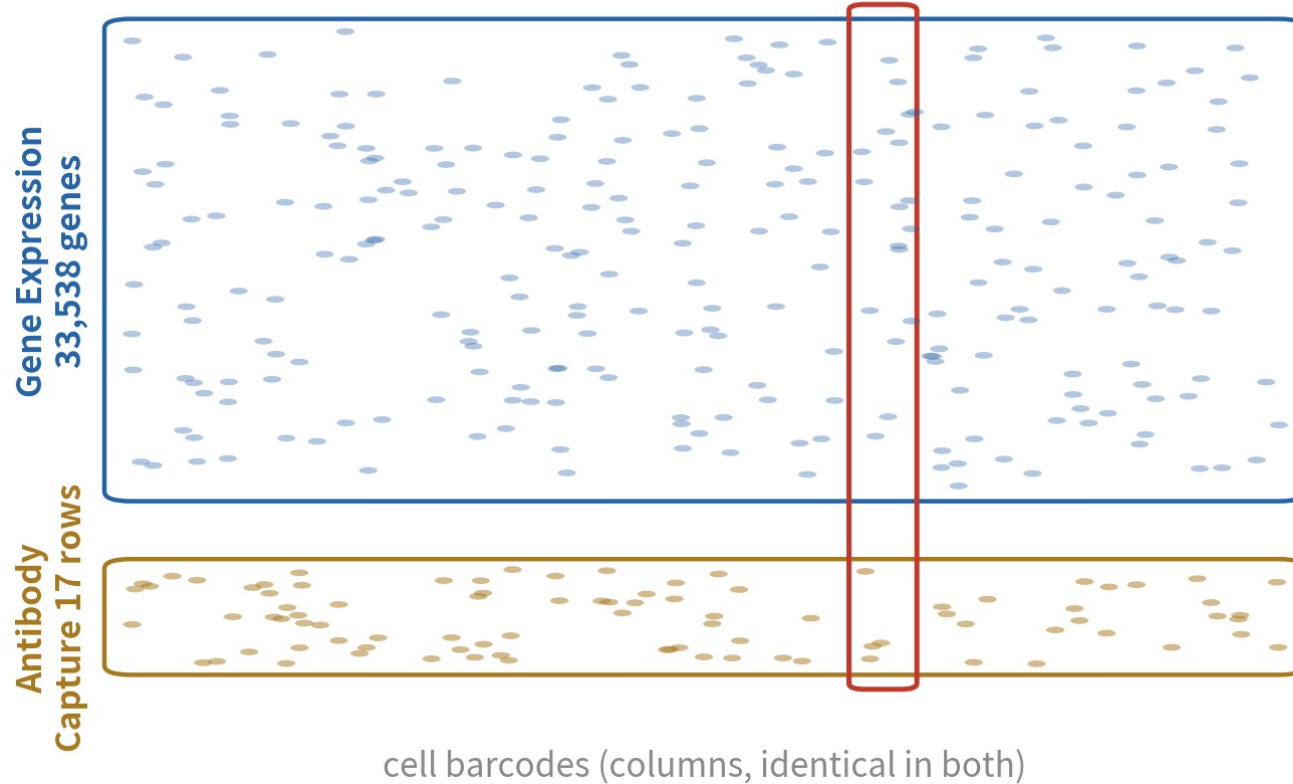
Anatomy of the feature reference

```
cat> feature_ref.csv << 'E0 F'  
id,name,read,pattern,sequence,feature_type  
CD3,CD3_TotalB,R2,5PNNNNNNNNNNN(BC),AACAAAGACCCTTGAG,Antibody Capture  
CD19,CD19_TotalB,R2,5PNNNNNNNNNNN(BC),CTGGGCAATTACTCG,Antibody  
Capture  
HT01,Hashtag1,R2,5PNNNNNNNNNNN(BC),GTCAACTCTTTAGCG,Antibody Capture  
HT02,Hashtag2,R2,5PNNNNNNNNNNN(BC),TGATGGCCTATTGGG,Antibody Capture  
E0 F
```

The sequence column is each antibody's ID — copy it from the vendor datasheet, verify character by character

The output matrix: one barcode, two feature types

one cell = one barcode, one column in each matrix



Gene Expression and Antibody Capture stack in one matrix, told apart by column 3 of features.tsv

column 3 of features.tsv

each row tagged Gene Expression
or Antibody Capture

Read10X in R

returns a named list:
two matrices, two names

into Seurat

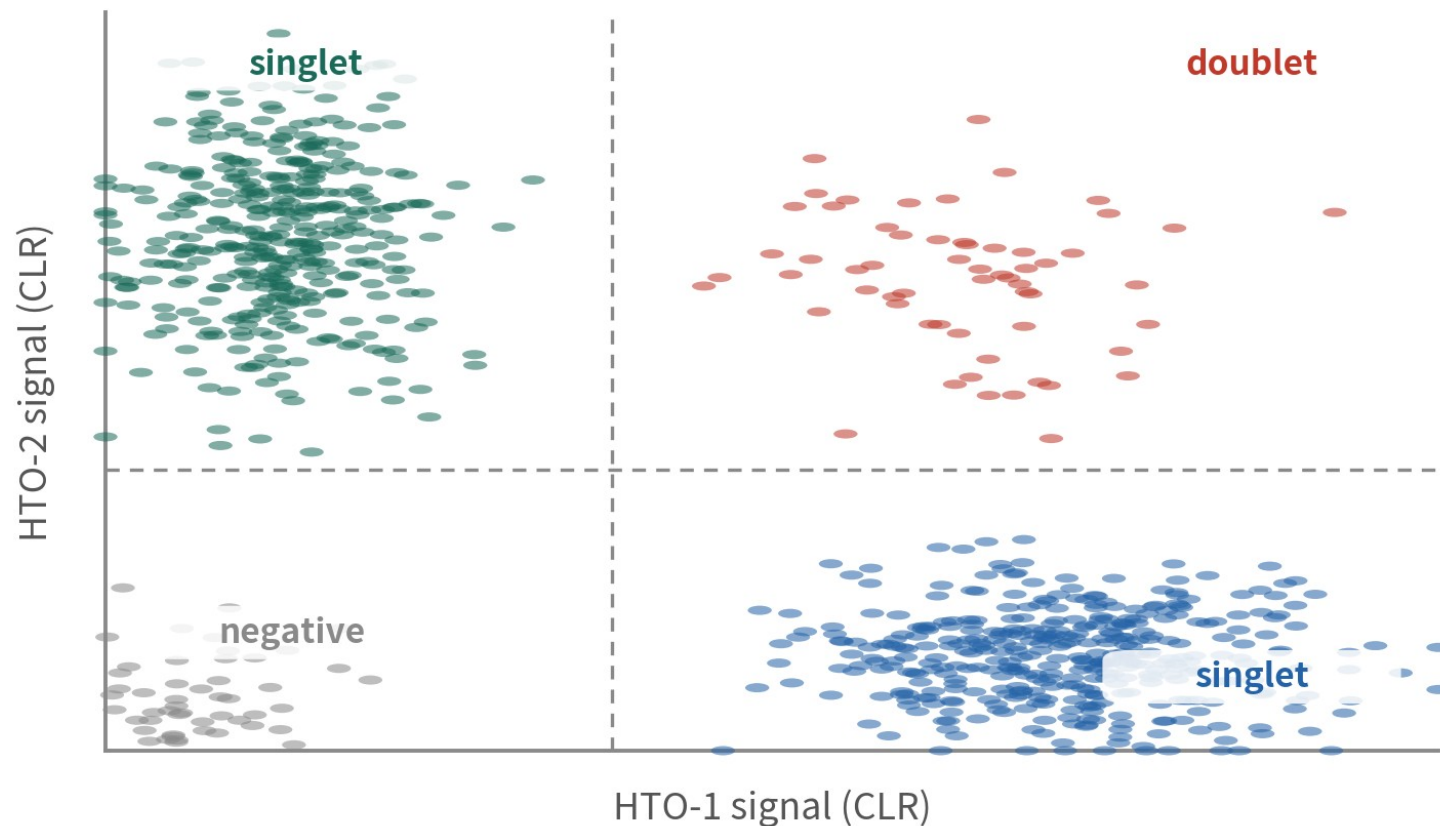
RNA assay + ADT assay,
one object, same cells

05

Hashing demux and the handoff to R

HTODemux runs in R — and how the matrix loads into Seurat

The three demux calls



singlet

one HTO high, rest low
→ clear origin, keep

doublet

two or more HTOs high
→ cross-sample doublet, drop

negative

all HTOs low
→ unstained; handle with care

simulated data

One HTO high = singlet; several high = doublet; all low = negative

Loading in Seurat: one object, two assays

```
library(Seurat)
set.seed(1234)

d <- Read10X("outs/filtered_feature_bc_matrix/")
names(d) # 有 FB 時回傳的是 list
pbmc <- CreateSeuratObject(counts = d[["Gene Expression"]])
pbmc[["HTO"]] <- CreateAssayObject(
  counts = d[["Antibody Capture"]])
pbmc
```

An object of class Seurat
33555 features across 7865 samples within 2 assays
Active assay: RNA (33538 features, 0 variable features)
1 other assay present: HTO

RNA and HTO live in one object, sharing the same cell barcodes

HTODemux: assigning cells to samples

```
pbm c <- Norm alizeData (pbm c, assay = "HT0 ",  
                        nom alization.m ethod = "CLR")  
pbm c <- HT0 Dem ux (pbm c, assay = "HT0 ",  
                    positive.quantile = 0.99)  
table (pbm c$HT0 _classification.g lobal)
```

Doublet	Negative	Singlet
680	346	6839

Calls land in metadata; the usual next step subsets the singlets before downstream work

What you have at this point

Demuxed samples



every cell knows which sample it came from

Two data layers



RNA plus protein (or tags) in one object

Next: B5



filtered matrix in hand
— downstream starts at QC

06

Where people get hurt

Three big traps, two small ones — consequences included

Trap 1: needing full depth after aggr

What it is

Run aggr with the default `--normalize=mapped`, then base all further analysis on the merged output.



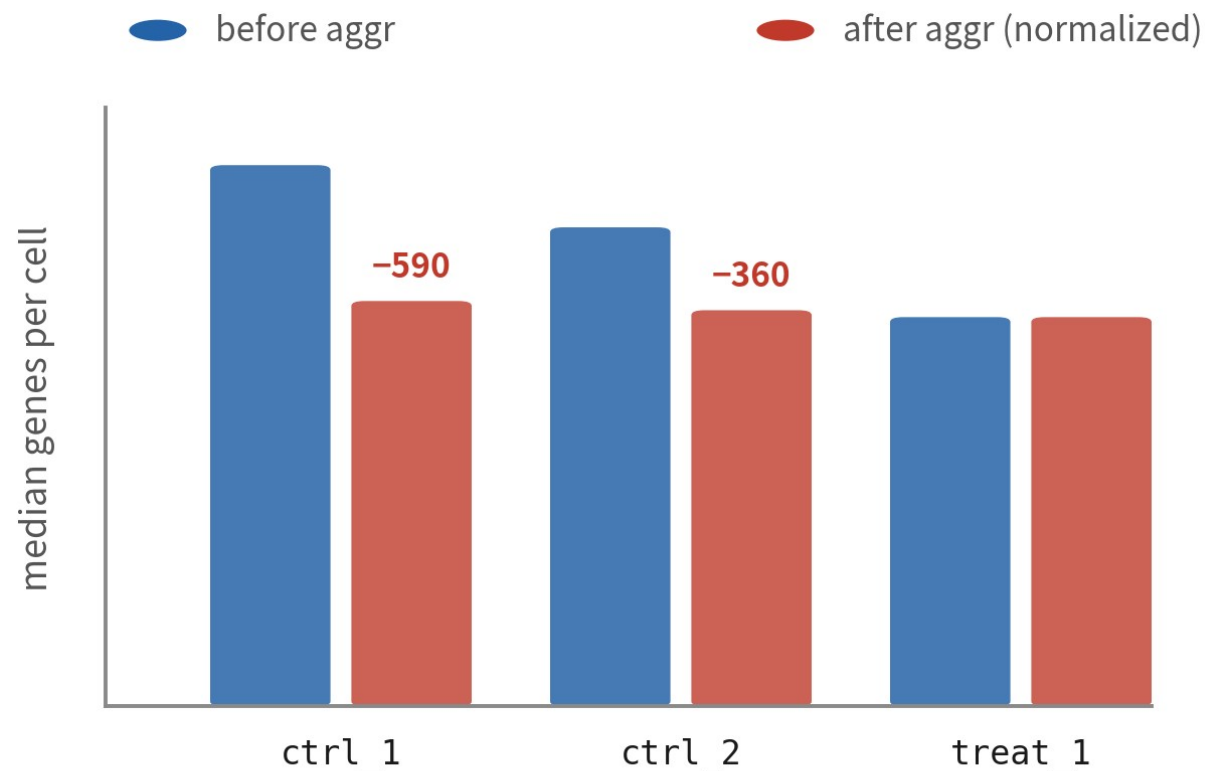
Why it happens

The default looks harmless — 'normalization' even sounds like a good thing.

What it costs you

Your deepest run loses near half its reads; saturation or rare-transcript work forces you back to the per-sample outputs.

Discarded reads do not come back



the discarded reads are in no output

saturation, rare transcripts, depth-sensitive work?

the aggr output cannot give them back —

you must return to each sample's count output

one-line check: the aggregation table in web_summary shows each sample's discard rate

simulated data

One-line check: aggr's web_summary tables each sample's discard fraction

Trap 2: a typo in the feature reference

What it is

A hand-typed sequence goes wrong — one base off, pasted on the wrong row, or taken from the wrong product line.



Why it happens

A 15-base string — the eye simply cannot proofread it.

What it costs you

Reads stop matching the reference and that ADT flatlines — while the pipeline finishes with zero errors.

One base kills the whole ADT

the CD3 sequence field in feature_ref.csv:

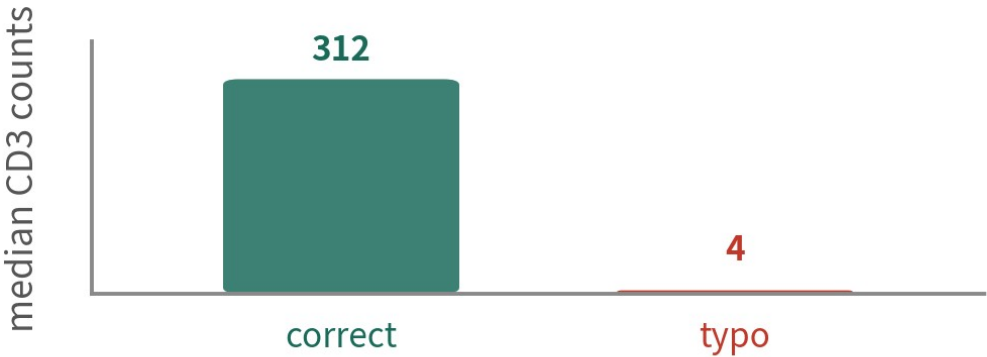
vendor datasheet

A A C A A G A C C C T T G A G

what you typed

A A C A A G A C C C T T G C G

one base off



reads no longer match the reference — the ADT flatlines and the pipeline still finishes without a single error

one-line check: a suspiciously low 'Antibody Reads Usable' in web_summary

simulated data

One-line check: a suspiciously low Antibody Reads Usable in web_summary

Trap 3: dropping every hashing negative

What it is

After HTODemux, subset out all negatives and never look back.



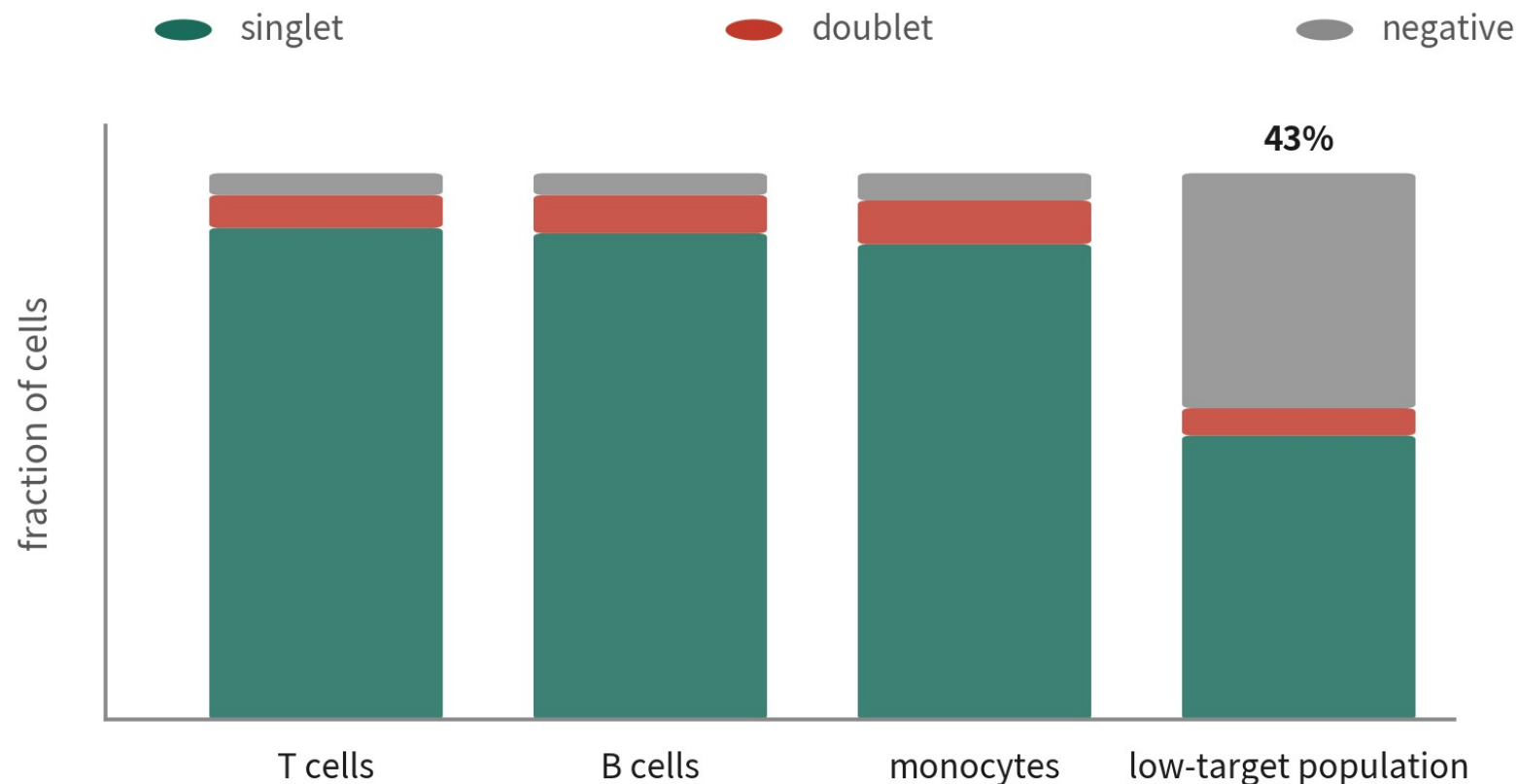
Why it happens

'No tag, no information' sounds perfectly reasonable.

What it costs you

Some populations are naturally low in the hashtag antibody's target — the whole group calls negative, and dropping it is a systematic bias.

Negatives are not uniformly distributed



the cost of dropping all negatives

hashtag antibodies bind surface proteins (CD298 / B2M) — cells low in the target stain poorly and land in negative

dropping them all = losing an entire population, systematically

simulated data

One-line check: `table(cell type, HTO call)` — negatives piling into one population is a warning

Two smaller traps, for the record

Trap	Consequence & symptom	Prevention
GEX / FB fastqs swapped	mapping rates crash on both sides	check the libraries CSV row by row
multi config missing [samples]	config validation fails before launch	let multi validate the config first

TRAPS, SUMMED UP

**Upstream mistakes rarely throw errors —
they just leave quietly wrong numbers.**

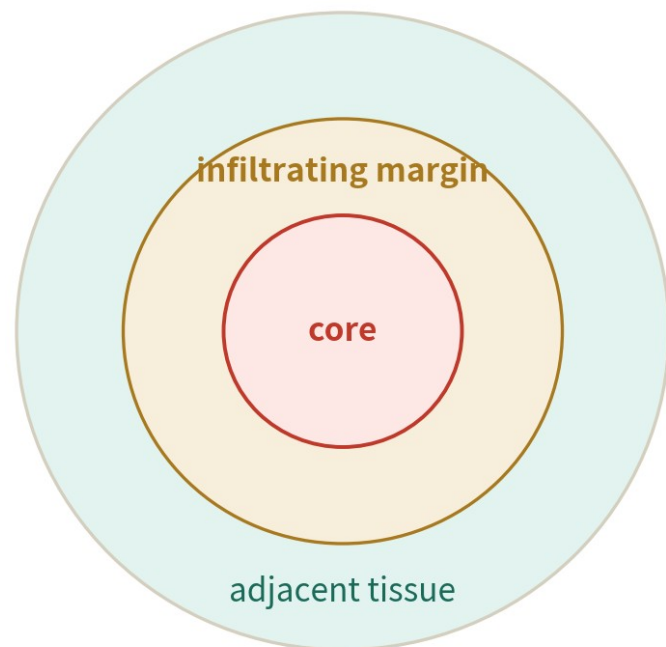
Three places in web_summary: the aggregation table, Antibody Reads Usable, per-sample cell counts. Check them every time.



Strategy Room: Complex Samples

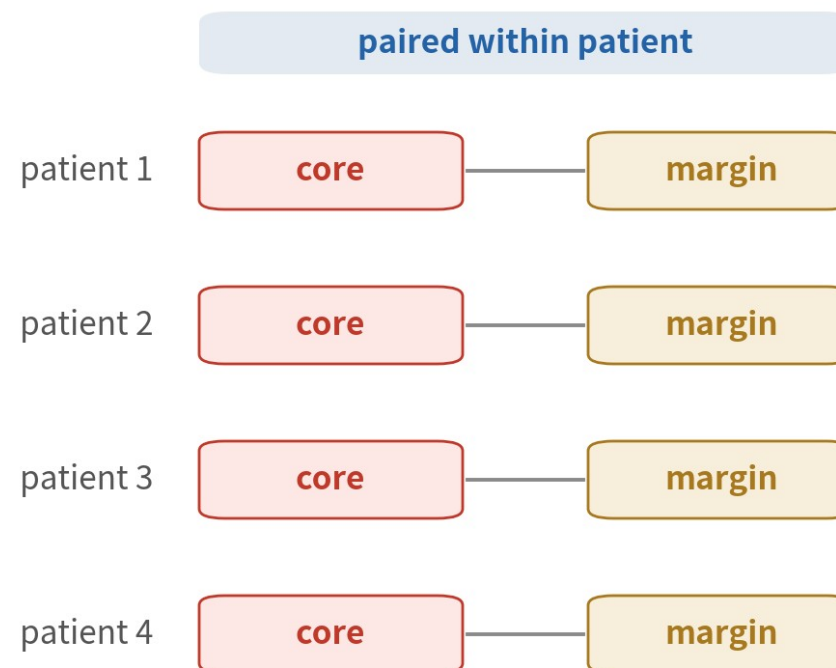
You know the upstream commands — now imagine you hold not four tubes of PBMC, but a tumor

Multi-region sampling: core, margin, pairing



**one tumor, three ecosystems —
one tube shows you only one**

GSE84465 (Darmanis 2017): four GBM patients × core / margin



8 samples • 3,589 cells • n = 4 pairs
pairing removes patient-to-patient noise

the design-time lesson

the margin holds only
31 malignant cells —
"malignant core vs margin"
cannot be asked

→ before surgery, estimate cells
per region × per type

the pairing bonus

same patient, same CNV —
merging is far simpler
than across patients (B11)

schematic; sample & cell numbers from GSE84465

GSE84465: four GBM patients × paired core/margin — only 31 malignant cells at the margin, a lesson for design time

In tumors, hashing's assumptions start to crack

Trap three's negative bias, amplified several-fold in tumors

Dissociation stress



solid tumors need long enzymatic digestion; surface proteins take damage and negatives rise across the board

Antigen expression varies



malignant CD298/ β 2M levels vary with patient and CNV — the bias lands exactly on the cells you care about

Frozen & snRNA



nuclei carry no surface proteins, so antibody hashing is out — use nucleus tags or genotype demultiplexing

Pre/post designs: block the confound at design time

Integration can fix uneven mixing — not perfect overlap

risky design

all pre now, all post later

- batch and treatment overlap perfectly
- B11's integration cannot tell them apart
- no statistics can rescue the design

design-time prevention

pair up, spread the batches

- analyse pre vs post paired within patient
- every batch carries both pre and post samples
- when possible, pool pairs in one channel — batch zeroed

Episode checklist



lanes go to count; samples each get their own count run



merge in R by default; aggr only when one matrix is required



when using aggr, decide --normalize deliberately



CellPlex or VDJ → multi; ADT/HTO → count+FB works too



feature reference sequences verified character by character



inspect the negatives before deciding their fate

Tick all six and you graduate from the upstream arc

ONE-LINE SUMMARY

**count runs a sample, multi runs an experiment —
and merging belongs downstream by default.**

If you can explain this line to a labmate, this episode has landed.

Check yourself 1

What does aggr's default `--normalize=mapped` do?

- A Rescale every cell's UMIs to the same total
- B Subsample every run's reads down to the shallowest run's depth
- C Remove batch effects
- D Append -1, -2 suffixes to barcodes

Answer B. It subsamples at the read level, dragging every run down to the shallowest depth; the discarded reads appear in no output. It is not batch correction, and the suffixes happen regardless of the normalize setting.

Check yourself 2

Which experiment must go through cellranger multi?

- A Four samples, each with its own count run, to be combined
- B GEX plus CITE-seq antibodies (ADT)
- C CellPlex (CMO) multiplexed samples
- D One library sequenced across four lanes

Answer C. Only multi can demultiplex CMO tags. A uses aggr or R-side merging, B works with count plus a libraries CSV, and D is handled by count automatically.

Check yourself 3

**After HTODemux, one cell type shows ten times the negative rate of the others.
The right response is?**

- A Negatives are meant to be dropped — drop them
- B Suspect that population stains poorly for the HTO antibody; investigate before deciding
- C Raise positive.quantile until negatives shrink
- D Rerun HTODemux

Answer B. Hashtag antibodies bind specific surface proteins; a population low in the target fails to stain as a block. Negatives concentrated in one type are a biological signal — dropping them or tuning them away just hides the bias.

NEXT EPISODE

B5: Loading data & QC

The upstream arc ends here — you hold a filtered matrix.

But what Cell Ranger calls a cell, Seurat may not accept. Next we enter R: which cells go, and who decides?